

## Leva.Framework.Sample.LocalIdentity

```
1  using Leva.Framework.Core;
2  using Leva.Framework.Identity;
3  using Leva.Framework.Identity.AspNet;
4  using Leva.Framework.Identity.Local;
5  using Leva.Framework.Identity.Memory;
6  using FrameworkAuthenticationService = Leva.Framework.Identity.AuthenticationService;
7  using FrameworkAuthorizationService = Leva.Framework.Identity.AuthorizationService;
8  using FrameworkSystemClock = Leva.Framework.Core.SystemClock;
9
10 var builder = WebApplication.CreateBuilder(args);
11
12 // Memory stores keep the sample self-contained.
13 var principalStore = new MemoryPrincipalStore();
14 var sessionStore = new MemoryAuthSessionStore();
15 var sessionService = new AuthSessionService(sessionStore);
16
17 // LocalPrincipalServices groups the local credential pieces used by the sample.
18 var localServices = LocalPrincipalServices.Create(principalStore);
19
20 // AuthenticationService uses the local-secret policy in this sample.
21 var authentication = new FrameworkAuthenticationService([localServices.AuthenticationPolicy],
22 sessionService);
23
24 // Authorization stays framework-level; ASP.NET Core adapts requests to it.
25 var authorization = new FrameworkAuthorizationService([new BuiltInAuthorizationPolicy()]);
26 var adminPermission = new PrincipalPermission("account.manage");
27
28 var admin = new Principal(
29     PrincipalId.New(),
30     "Local Admin",
31     "admin@example.test",
32     Roles: new HashSet<PrincipalRole> { new("admin") },
33     Permissions: new HashSet<PrincipalPermission> { adminPermission }
34 );
35 var operatorUser = new Principal(
36     PrincipalId.New(),
37     "Local Operator",
38     "operator@example.test",
39     Roles: new HashSet<PrincipalRole> { new("operator") }
40 );
41
42 // Admin can access the protected endpoint; operator can log in but cannot.
43 principalStore.Add(admin, "admin");
44 principalStore.Add(operatorUser, "operator");
45 await localServices.CredentialService.CreateAsync("admin", "password", admin.Id);
46 await localServices.CredentialService.CreateAsync("operator", "password", operatorUser.Id);
47
48 // Register the composed framework services for the ASP.NET adapters.
49 builder.Services.AddSingleton<IClock, FrameworkSystemClock>();
50 builder.Services.AddSingleton<IPrincipalStore>(principalStore);
51 builder.Services.AddSingleton<ILocalCredentialStore>(localServices.Credentials);
52 builder.Services.AddSingleton(localServices.CredentialService);
53 builder.Services.AddSingleton(localServices.AuthenticationPolicy);
54 builder.Services.AddSingleton<IAuthSessionStore>(sessionStore);
55 builder.Services.AddSingleton(sessionService);
56 builder.Services.AddSingleton(authentication);
57 builder.Services.AddSingleton(authorization);
58
59 // Adapt framework identity services to ASP.NET Core auth middleware.
60 builder.Services.AddAspNetPrincipalServices(options =>
61 {
62     options.AllowHeaderSession = true;
63     options.SecureCookie = false;
64 });
```

```

65 // ASP.NET Core owns the policy; the requirement uses framework permission concepts.
66 builder.Services.AddAuthorization(options =>
67 {
68     options.AddPolicy("AccountManage",
69         policy =>
70             policy.Requirements.Add(
71                 new
72                 AspNetAuthorizationRequirement(AuthorizationRequirement.Permission(adminPermission))
73             );
74 });
75
76 var app = builder.Build();
77 app.UseDefaultFiles();
78 app.UseStaticFiles(new StaticFileOptions
79 {
80     OnPrepareResponse = context =>
81     {
82         // Samples change often while developing, so disable browser caching for local static
83         files.
84         context.Context.Response.Headers.CacheControl = "no-store, no-cache, must-
85         revalidate";
86         context.Context.Response.Headers.Pragma = "no-cache";
87         context.Context.Response.Headers.Expires = "0";
88     },
89 });
90 // Middleware reads the framework session cookie and populates HttpContext.User.
91 app.UseAuthentication();
92 app.UseAuthorization();
93
94 app.MapGet("/favicon.ico", () => Results.NoContent());
95
96 // Helper endpoint clears the framework auth-session cookie for repeatable checks.
97 app.MapPost("/sample/logout",
98     (HttpContext context) =>
99     {
100         context.Response.Cookies.Delete(AspNetDefaults.SessionCookieName);
101         return Results.Ok(new { cleared = true });
102     }
103 );
104
105 // Expose framework login and current-principal endpoints.
106 app.MapAspNetIdentityEndpoints();
107
108 // Prove that ASP.NET Core middleware can enforce a framework permission.
109 app.MapGet("/account/admin-area",
110     (HttpContext context) =>
111         Results.Ok(new
112             {
113                 message = $"Hello {context.User.Identity?.Name ?? "unknown"}.",
114                 isAuthenticated = context.User.Identity?.IsAuthenticated == true,
115                 requiredPermission = "account.manage",
116             }
117         )
118     )
119     .RequireAuthorization("AccountManage");
120
121 app.Lifetime.ApplicationStarted.Register(() =>
122 {
123     Console.WriteLine("LocalIdentity sample");
124     Console.WriteLine("Open the printed local URL and click Run all checks, or use the curl commands from
125     the sample README.");
126 });
127 app.Run();
128

```